Introdução a Algoritmos

Aula 10 – Strings

Raoni F. S. Teixeira · 1s/2026

Objetivos desta aula
Ao final desta aula, você deverá ser capaz de:

1. Explicar o papel do terminador '0' e calcular o tamanho de buffer necessário para armazenar uma string de N caracteres;
2. Distinguir char s[] de char *s, identificando qual pode ser modificado e por que tentar modificar um literal causa falha;
3. Percorrer uma string caractere a caractere usando o terminador como condição de parada, e implementar strlen do zero;
4. Usar corretamente strcmp para comparar strings, explicando por que

   0.1 não pode ser usado para essa finalidade.

1 O problema de representar texto

Até agora, trabalhamos com números — inteiros, reais, caracteres isolados. Mas a maioria dos programas precisa manipular texto: nomes, mensagens, comandos, arquivos. Como representar a palavra "algoritmo" em C?

A resposta mais direta seria guardar cada letra numa variável `char`:

```
char c1 = 'a', c2 = 'l', c3 = 'g', c4 = 'o', c5 = 'r',
     c6 = 'i', c7 = 't', c8 = 'm', c9 = 'o';
```

Mas isso não escala — e não há como passar "o texto" para uma função sem passar nove variáveis separadas. Um vetor de `char` resolve o problema de armazenamento:

```
char palavra[] = {'a','l','g','o','r','i','t','m','o'};
```

Mas agora surge outro problema: se `printf` receber esse vetor, como ela sabe onde o texto termina? O vetor tem 9 elementos, mas `printf` não sabe disso — ela só recebe o endereço do primeiro byte.

A solução adotada em C é elegante e simples: marcar o fim do texto com um caractere especial, o **terminador nulo** `'\0'`. Toda string em C termina com esse byte de valor zero. É essa convenção que permite que funções como `printf`, `strlen` e `strcpy` funcionem sem precisar de um parâmetro de tamanho.

2 String como vetor de char com terminador

Definição — String em C
Uma string é um vetor de `char` cujo último elemento significativo é o caractere nulo `'\0'` (valor ASCII 0). O terminador marca o fim do conteúdo textual — tudo após ele é ignorado pelas funções de string.

A forma mais comum de criar uma string é com um **literal de string** entre aspas duplas:

```
char nome[] = "algoritmo";
```

O compilador cria automaticamente um vetor com os 9 caracteres da palavra **mais o terminador nulo** ao final — 10 bytes no total:

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	Tam.
'a'	'l'	'g'	'o'	'r'	'i'	't'	'm'	'o'	**'\0'**	10

A posição `[9]` (em vermelho) é o terminador nulo — **não faz parte do texto**, mas é indispensável para demarcar seu fim.

2.1 O espaço adicional

Essa é a regra mais importante e mais esquecida ao declarar strings em C:

Para guardar uma string de N caracteres, o vetor precisa ter tamanho mínimo de N + 1 — o "+1" é o espaço para o terminador `'\0'`.

```
char nome[10];   /* guarda no maximo 9 caracteres + '\0' */
char cpf[12];    /* guarda "000.000.000-0" (11 chars) + '\0' */
```

Declarar `char nome[9]` para guardar "algoritmo" (9 letras) é um erro: não há espaço para o `'\0'` e o programa terá comportamento indefinido.

3 Declaração e inicialização

Há três formas principais de declarar strings em C, cada uma com características distintas.

3.1 Vetor com tamanho explícito

```
char nome[50];              /* buffer de 50 chars, conteudo indeterminado */
char cidade[20] = "Cuiaba"; /* inicializado; restante preenchido com '\0' */
```

Esta é a forma mais comum para strings que serão lidas do usuário ou modificadas. O tamanho define o buffer máximo disponível.

3.2 Vetor sem tamanho (deduzido pelo compilador)

```
char saudacao[] = "Ola, mundo!";   /* compilador infere tamanho 12 */
```

Conveniente quando o conteúdo é fixo e conhecido em tempo de compilação. O vetor pode ser modificado — seus elementos são variáveis normais.

3.3 Ponteiro para literal — `char *s`

```
char *msg = "Erro: valor invalido";
```

Esta forma **parece** equivalente à anterior, mas tem uma diferença fundamental: o literal `"Erro: valor invalido"` é armazenado numa região de memória somente-leitura do programa. O ponteiro `msg` aponta para lá, mas **não** para um buffer modificável.

Atenção

Tentar modificar uma string apontada por `char *` é comportamento indefinido — na maioria dos sistemas, causa uma falha imediata:

```
char *msg = "Ola";
msg[0] = 'o';    /* ERRADO — modifica memoria somente-leitura */

char buf[] = "Ola";
buf[0] = 'o';    /* correto — buf e um vetor modificavel */
```

Use `char *` para strings constantes que nunca serão alteradas. Use `char[]` para strings que precisam ser modificadas ou preenchidas.

A tabela abaixo resume as diferenças práticas:

Aspecto	`char s[]`	`char *s`
Onde vive	Na pilha (stack) — memória local da função	Ponteiro na pilha; literal no segmento de dados read-only
Modificável?	Sim — cada `s[i]` é uma variável normal	Não — tentar modificar é comportamento indefinido
Tamanho	Fixo em tempo de compilação	O ponteiro pode ser redirecionado para outra string
Uso típico	Buffers para leitura, strings processadas	Mensagens fixas, constantes textuais

Ponta do fio

Por que `char *s = "texto"` aponta para memória somente-leitura? Porque literais de string são armazenados no segmento de código do executável, junto com as instruções do programa. O sistema operacional mapeia esse segmento como somente-leitura por razões de segurança e eficiência. O mecanismo completo — segmentos de memória, tabela de símbolos, carregamento do executável — é tema de uma aula futura sobre ponteiros avançados.

4 Leitura e escrita

4.1 Escrita com printf

O especificador `%s` imprime uma string até encontrar `'\0'`:

```
char nome[] = "Maria";
printf("Nome: %s\n", nome);          /* Nome: Maria        */
printf("Nome: %10s\n", nome);        /* Nome:      Maria   (campo de 10) */
printf("Nome: %-10s|\n", nome);      /* Nome: Maria     |  (alinhado esq) */
```

4.2 Leitura com scanf

O especificador `%s` lê uma sequência de caracteres até encontrar espaço em branco (espaço, tab, newline) e adiciona `'\0'` automaticamente:

```
char nome[50];
scanf("%s", nome);   /* note: sem & — nome ja e um endereco */
```

Diferente de variáveis simples, **não usamos `&`** com vetores de char no `scanf` — o nome do vetor já é o endereço do primeiro elemento.

> **Atenção**
> `scanf("%s", nome)` não verifica se a entrada cabe no buffer. Se o usuário digitar mais de 49 caracteres, o programa escreverá além do vetor — um **buffer overflow**. A forma correta limita o número de caracteres lidos:
>
> ```
> char nome[50];
> scanf("%49s", nome);   /* le no maximo 49 chars + '\0' */
> ```
>
> O número dentro do especificador é sempre `tamanho_do_buffer - 1`.

4.3 Leitura de linha completa com fgets

`scanf("%s")` para no primeiro espaço — não lê frases. Para ler uma linha inteira (incluindo espaços), use `fgets`:

```
char frase[100];
fgets(frase, sizeof(frase), stdin);
```

`fgets` lê até `sizeof(frase) - 1` caracteres ou até encontrar `'\n'`, o que vier primeiro, e sempre adiciona `'\0'`. Diferente de `scanf`, ela **inclui o `'\n'`** no buffer quando a linha cabe — isso deve ser tratado se indesejado.

5 Percorrendo uma string: as três perguntas

Uma string é um vetor, então percorrê-la usa exatamente o mesmo esqueleto da Aula 9. Mas há duas formas de controlar o laço — com índice inteiro ou testando diretamente o terminador:

```
/* Forma 1: indice inteiro com strlen */
for (int i = 0; i < strlen(s); i++) {
    /* opera sobre s[i] */
```

```
}

/* Forma 2: testar o terminador diretamente */
for (int i = 0; s[i] != '\0'; i++) {
    /* opera sobre s[i] */
}
```

Aplicando as três perguntas à Forma 2:

- **O que se repete?** — a operação sobre um caractere.
- **Quantas vezes?** — enquanto o caractere atual não for `'\0'`.
- **O que muda?** — o índice `i`, que seleciona o próximo caractere.

A Forma 2 é mais direta: ela diz exatamente o que a string em C significa — processar caracteres **até encontrar o terminador**. A Forma 1 é equivalente, mas chama `strlen` a cada iteração (ineficiente para strings longas).

Exemplo — Implementando strlen do zero

Entender `strlen` é entender strings. Sua implementação é um laço de três linhas:

```
int meu_strlen(char s[]) {
    int i = 0;
    while (s[i] != '\0')
        i++;
    return i;   /* i e o numero de caracteres antes do '\0' */
}
```

Para `s = "algoritmo"`: o laço percorre os índices 0 a 8 (nove caracteres), para quando encontra `'\0'` em `s[9]`, e retorna 9. O terminador **não é contado** — `strlen` retorna o comprimento do texto, não do buffer.

6 Funções de string.h

A biblioteca padrão `<string.h>` oferece as operações mais comuns sobre strings. Apresentamos cada uma junto com sua implementação manual — não para que você reimplemente, mas para que o comportamento não seja uma caixa preta.

6.1 strlen — comprimento

```
size_t strlen(const char *s);
```

Retorna o número de caracteres antes do `'\0'`. Já vimos sua implementação acima.

6.2 strcpy — copiar

```
char *strcpy(char *dest, const char *src);
```

Copia `src` para `dest`, incluindo o `'\0'`. Implementação:

```
void meu_strcpy(char dest[], char src[]) {
    int i = 0;
    while (src[i] != '\0') {
        dest[i] = src[i];
        i++;
    }
```

```
    dest[i] = '\0';   /* copia o terminador */
}
```

Atenção
`strcpy` não verifica se `dest` tem espaço suficiente. Se `src` for maior que o buffer de `dest`, ocorre buffer overflow. Para código seguro, use `strncpy(dest, src, n)` que copia no máximo `n` caracteres.

6.3 strcat — concatenar

```
char *strcat(char *dest, const char *src);
```

Acrescenta `src` ao final de `dest`. Isso significa encontrar o `'\0'` de `dest` e começar a copiar `src` a partir daí:

```
void meu_strcat(char dest[], char src[]) {
    int i = 0, j = 0;
    while (dest[i] != '\0') i++;   /* avanca ate o fim de dest */
    while (src[j]  != '\0')        /* copia src a partir dali  */
        dest[i++] = src[j++];
    dest[i] = '\0';
}
```

O buffer de `dest` deve ter espaço para `strlen(dest) + strlen(src) + 1` caracteres — responsabilidade do programador.

6.4 strcmp — comparar

```
int strcmp(const char *s1, const char *s2);
```

Compara `s1` e `s2` lexicograficamente. Retorna:

- 0 se as strings são iguais;
- valor negativo se `s1` vem antes de `s2`;
- valor positivo se `s1` vem depois de `s2`.

```
void meu_strcmp_simplificado(char s1[], char s2[]) {
    int i = 0;
    while (s1[i] != '\0' && s2[i] != '\0' && s1[i] == s2[i])
        i++;
    return s1[i] - s2[i];   /* 0 se iguais, negativo/positivo caso contrario */
}
```

A armadilha mais comum com strings: usar == para comparar.

```
char s1[] = "ola";
char s2[] = "ola";

if (s1 == s2)          /* ERRADO: compara enderecos, nao conteudo */
    printf("iguais\n");

if (strcmp(s1, s2) == 0)   /* correto */
    printf("iguais\n");
```

> `s1 == s2` compara os **endereços** dos dois vetores — que são diferentes, mesmo que o conteúdo seja idêntico. O resultado é sempre falso para dois vetores distintos, independentemente do texto. Strings em C nunca se comparam com ==.

7 Exemplo integrador: verificador de palíndromo

Um palíndromo é uma palavra que se lê igual de frente para trás: “arara”, “radar”, “level”. O programa abaixo verifica se uma palavra é palíndromo, usando as operações de string que aprendemos.

```
#include <stdio.h>
#include <string.h>

/*
 * Retorna 1 se s e palindromo, 0 caso contrario.
 * Pre-cond.: s e uma string valida (terminada em '\0')
 */
int palindromo(char s[]) {
    int esq = 0;
    int dir = strlen(s) - 1;

    while (esq < dir) {
        if (s[esq] != s[dir])
            return 0;
        esq++;
        dir--;
    }
    return 1;
}

/*
 * Inverte s no proprio buffer.
 */
void inverte(char s[]) {
    int esq = 0;
    int dir = strlen(s) - 1;
    while (esq < dir) {
        char temp = s[esq];
        s[esq] = s[dir];
        s[dir] = temp;
        esq++;
        dir--;
    }
}

int main() {
    char palavra[100];
    printf("Digite uma palavra: ");
    scanf("%99s", palavra);

    printf("Comprimento: %d\n", (int)strlen(palavra));

    if (palindromo(palavra))
        printf("E palindromo.\n");
    else
```

```
        printf("Nao e palindromo.\n");

    inverte(palavra);
    printf("Invertida: %s\n", palavra);

    return 0;
}
```

Observe como `palindromo` e `inverte` usam dois índices caminhando em direções opostas — o mesmo padrão do exercício de inversão de vetor da Aula 9, agora aplicado a caracteres. A única diferença é que o tamanho vem de `strlen` em vez de um parâmetro `n`.

8 Resumo: o que toda string em C pressupõe

Toda vez que você declara ou manipula uma string em C, três invariantes devem ser mantidas:

1. **O buffer tem espaço para o texto mais `'\0'`**. Sempre declare com `N + 1` posições para texto de comprimento N.
2. **O terminador está presente e na posição correta**. Funções de `<string.h>` e `printf("%s")` dependem disso. Construir uma string manualmente char a char exige colocar `'\0'` explicitamente ao final.
3. **Nunca escreva além do buffer**. C não verifica limites. Um caractere a mais sobrescreve memória de outra variável — ou de parte do programa.

9 Exercícios

1. Escreva uma função `int conta_vogais(char s[])` que retorna o número de vogais (a, e, i, o, u — maiúsculas e minúsculas) numa string.
2. Escreva uma função `void maiusculas(char s[])` que converte todos os caracteres de `s` para maiúsculas **no próprio vetor**. Use o fato de que para qualquer letra minúscula, `c - 32` dá a maiúscula correspondente em ASCII. *(Dica: verifique se o caractere está entre `'a'` e `'z'` antes de converter.)*
3. Escreva uma função `int contem(char s[], char sub[])` que retorna 1 se `sub` aparece como substring de `s`, e 0 caso contrário. *(Dica: para cada posição `i` de `s`, verifique se os caracteres `s[i], s[i+1], ...` coincidem com `sub[0], sub[1], ...` até o `'\0'` de `sub`.)*
4. O programa abaixo tem três erros distintos envolvendo strings. Identifique cada um e corrija:

```
#include <stdio.h>
#include <string.h>
int main() {
    char *s1 = "Ola";
    char s2[3];
    strcpy(s2, s1);
    s1[0] = 'o';
    if (s1 == s2)
        printf("iguais\n");
    return 0;
}
```

5. *(Desafio)* Escreva uma função `void remove_espacos(char s[])` que remove todos os espaços de uma string **no próprio buffer**, sem usar um vetor auxiliar. Após a função, `"ola mundo"` deve se tornar `"olamundo"`. *(Dica: use dois índices — um para leitura e outro para escrita, ambos percorrendo o mesmo vetor.)*